Empowering Privacy: A Zero Cost Protocol for Concealing LGBTQ Search Queries

Akshit Aggarwal[1], Pulkit Bharti[1], Yang Li[2], and Srinibas Swain[1,3]

[1] Indian Institute of Information Technology, Guwahati, India
{akshit.aggarwal, pulkit.bharti21b, srinibas}@iiit.ac.in
[2] Deakin University, Melbourne, Australia
kelvin.li@deakin.edu.au
[3] School of Computer and Mathematical Sciences, University of Adelaide, Australia
srinibas.swain@adelaide.edu.au

Abstract. FHE-based private information retrieval (PIR) is widely used to maintain the secrecy of the client queries in a client-server architecture. There are several ways to implement FHE-based PIR. Most of these approaches results in server computation overhead. Attempts for reducing the server computation overhead results in 1) fetching incorrect results, 2) leakage of queries, 3) large number of homomorphic operations (which is a time consuming process), and 4) downloading the entire dataset in the client side. In this work, we design a three server based approach where the first server discuss the nature of dataset, second server stores the computation results performed over first server, and third server stores the dataset in accordance to the proposed novel technique, that is, restricted bin packing algorithm (RBA). The proposed three server based approach optimise the aforementioned limitations. Later we implement the designed protocol using Tenseal library. Our protocol provides to retrieve the data by providing security to the client's query.

Keywords: Homomorphic encryption · LGBTQ · Restricted Bin Packing (RBA) · PIR · Query · Recommendation.

1 Introduction

Due to extensive use of cloud applications, searching data from cloud has become ubiquitous. Over the last decade, many businesses have shifted their corporate data centres to cloud services like Microsoft Azure, Google Cloud, AWS, and many more. However, searching and retrieving these data from cloud comes with numerous attacks [1,2,3]. A long line of work [4,5,6,7,8] addresses the privacy concerns when a client wants to retrieve some data items from the cloud whenever the data is in encrypted form. However, none of these work discusses the privacy concern whenever a client wants to retrieve some data items from publicly available cloud (such as Wikipedia, Netflix, Google, and many more). Thus, there is a need for a strong mechanism that can privately retrieve the data from publicly available cloud.

Private information retrieval (PIR) is a mechanism that allows a client to retrieve the data items from cloud without revealing which item is being retrieved. PIR is widely used in many areas, such as private location delivery [9,10], anonymous communication [11,12], private voting system [13,14], privacy preserving recommendation services [15], and many more. PIR is divided into two categories, that is, information theoretic PIR, and computational PIR. In information theoretic PIR, non colluding multi servers are required where each server is designed for some distinct task. Computational PIR works on single server but requires some hard cryptographic assumptions (like factoring large number [16] or discrete logarithmic [17]). Due to these cryptographic assumptions, server needs high computational power to solve these assumptions and moreover requires financial overhead for answering client's query [18]. Thus, in this work, we use multi server based PIR approach as it is more budget friendly and does not require any cryptographic assumptions.

1.1 Related Work

In this section, we discuss the relevant literature in the area of private information retrieval.

To the best of our knowledge, PIR was first introduced by Chor et al. [19] by considering a multi-server-based approach where replicas of data were stored on each server. Later on, Kushilevitz et al. [20] introduced the first single server-based PIR protocol. The main limitation of previous works was high communication complexity [19,20,21]. To address the aforementioned limitation, Yi et al. [22] proposed a Fully Homomorphic Encryption (FHE)-based PIR scheme, achieving logarithmic communication complexity with respect to database size. However, the main limitation of their work is high computational overhead on the server. Reduction on server computation overhead comes with many drawbacks [23,24,25,26]. A few of them are discussed as follows.

First, a few authors have proposed batch-based encoding scheme (as batch encoding helps to apply multiple operations simultaneously, which reduces the server computation overhead) where the authors send multiple queries in a single slot [23,24,27]. The main limitation of their work is lack of ensurance of correctness of query result (as batching includes packing and unpacking of ciphertexts, which may lose some of the functionality of either query or database) [24,27]. To improve it, a few authors have proposed a multi-server-based approach to achieve correct query results [26,28]. The work by Ahmad et al. [26] proposed three servers (ideally interaction rounds between client and server in [26]) protocol where each server is responsible for distinct tasks and retrieves the relevant document. The main drawback of [26]'s scheme is oblivious transfer (where receiver gets one of many options secretly) that leaks the query access pattern. Gibbs et al. [28] proposed an adaptive query (which depends on the response of the previous query)-based approach where server is divided into a few chunks (ideally multi-servers) with query leakage problems. Later, a few authors proposed a single server-based approach to overcome the multi-server issues. The work by [29,25,30] performs various steps with the help of single

server. The work in [29] used Fermat's Little Theorem to convert each vector into either an encryption of one or zero, which is helpful for retrieving the record corresponding to the encryption of one. In contrast, [25] used asymmetric scalar-product-preserving encryption [4], which is helpful for retrieving the top most similar documents. Additionally, [30] used additive sharing of random vectors, providing authentication to the server. The main limitation of these works is high number of homomorphic operations (which is time consuming). Further, a few authors first download the database and then search for the desired query on the database, which increases the client storage [31,32,33]. While downloading the database, to turn the communication channel secure, the work by [32,33] introduces learning with error (LWE) based assumption [34], which introduces noise in the database which is redundant. Thus, reducing server computation faces the following challenges: 1) inaccurate query results, 2) query leakage, 3) a large number of homomorphic operations, and 4) increased client storage requirements.

1.2 Problem Formulation

Consider a server that holds a set of publicly available web links, where each link discusses the category of L/G/B/T/Q. There is a client who belongs to any of these categories who wishes to participate in a gender specific event. Thus, client needs some information on LGBTQ (which is available in links present on the server). Thus, due to privacy concerns, client wants to privately retrieve relevant web links from the server that are related to client's query.

1.3 Our Solution

First, we assign a fixed number to each category. Then, we propose three servers-based approach. The first server contains a set of digits representing the nature of each link (indicating whether the link discusses topics related to L, G, B, T, or Q), and web link location (it indicates the specific location of the link stored on the third server). Second server stores the computation results that is performed over the first server. Third server stores each link (to reduce the memory wastage, we arrange the web links according to restricted bin packing algorithm (RBA) (as discussed in Algorithm 3) (as shown in Fig. 1).
As we have multiple categories, and each link on Server 1 can potentially discuss any of these categories. Therefore, to align with the dimensions of Server 1, the client first expands their query by replicating it for each category (particularly five in our case), and sends it to the server. The server performs homomorphic computation over the query vector, encrypts the location (note that the location can be encrypted using any secure encryption scheme, such as Advanced Encryption Standard (AES), which is beyond the scope of this paper), and sends it to Server 2. Server 2 performs homomorphic decryption on the computed results and forwards the encrypted location to the client if the decryption result is zero. Finally, client homomorphically searches the location on the third server and retrieves the desired web link(s).

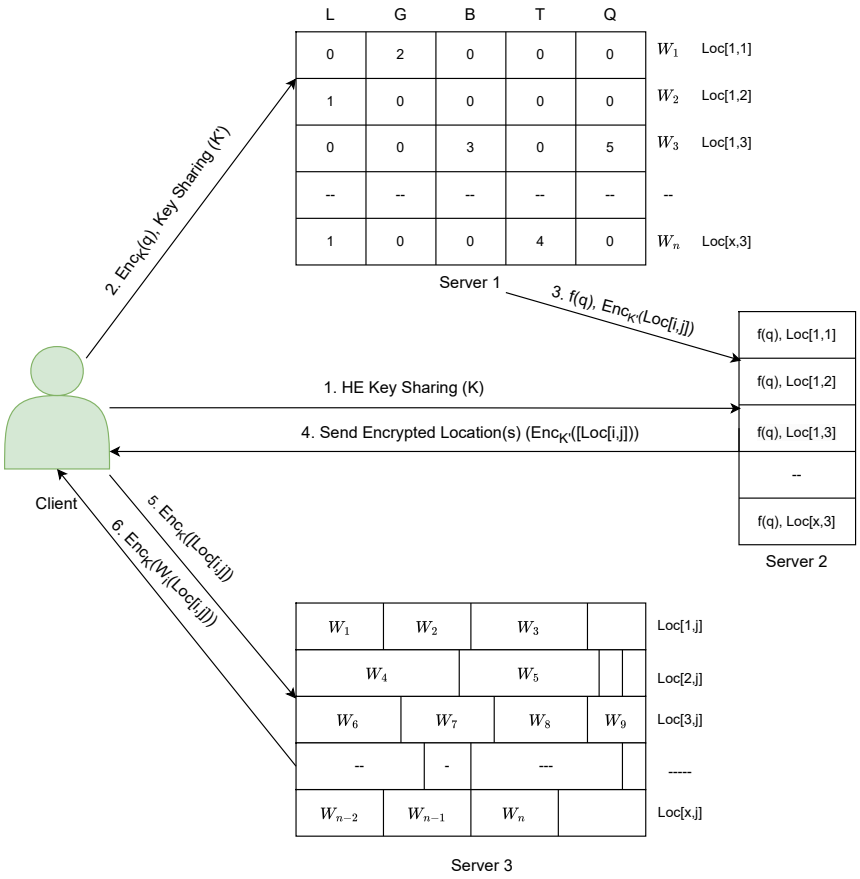

Fig. 1. Here K' represents secret key of any secure encryption algorithm, K represents homomorphic encryption key. Digits "1", "2", "3", "4", and "5" represent the presence of categories "L", "G", "B", "T", and "Q" in Server 1 respectively. W_k (where $1 \leq k \leq n$) represents web link number. $Loc[i,j]$ represents location (where link W_k is stored in j^{th} slot of bin i, (where $i,j > 0$)). q represents the client query. Server 2 stores the computation results obtained from first server. $Enc_{K'}(Loc[i,j])$ represents encryption of location using any secure encryption algorithm, and $Enc_K(q_i)$ represents homomorphic encryption of query q_i. x represents total number of locations (where $1 \leq x < n$ (where n is total number of web links)).

1.4 Our Contribution

We propose an LGBTQ query protocol to retrieve web links without revealing client's identity.

– First, three server based approach is used. In third server, we propose a novel restricted bin packing algorithm to arrange the web links (that helps to reduce the wastage).
– Second, we implement our protocol using TenSeal library that retrieves the desired web link(s) corresponding to query q.

1.5 Organization of our paper

In Section 2, we discussed the preliminaries used in our work. The proposed methodology of this work is discussed in Section 3. Section 4 discussed the implementation details of our work. Later, we identify the security and correctness of this work in Section 5. We discuss the limitations of our work in Section 6. Finally, the conclusion of our work is shown in Section 7.

2 Preliminary

In this section, we first discuss BFV homomorphic encryption which is used for encryption and decryption of client's query. Thereafter, we discuss the bin packing algorithm which is used for storage of data.

2.1 BFV Homomorphic Encryption

BFV encryption enables simultaneous operations on a data vector and leverages the Single Instruction Multiple Data (SIMD) programming model. BFV encryption mechanism adds noise whenever a plaintext is converted into ciphertext (where plaintext is an element from $\mathbb{Z}_p$ (p is prime number)). The amount of noise increases as homomorphic operations are performed on the encrypted data [35].
BFV encryption uses the following operations to convert plaintext into ciphertext, with all operations maintaining the plaintext under modulo p.

– **Add**(ct_1,ct_2) consists of two ciphertexts that contain the encryption of plaintext m_1, m_2, and returns an encryption of $(m_1 + m_2)$ (component-wise addition).
– **AddPlain**(ct_1,m_2) consists of ciphertext (encryption of m_1), plaintext m_2, and returns an encryption of $(m_1 + m_2)$ (component-wise addition).
– **Subtract**(ct_1,ct_2) consists of two ciphertexts that contain the encryption of plaintext m_1, m_2, and returns an encryption of $(m_1 - m_2)$ (component-wise subtraction).
– **SubtractPlain**(ct_1,m_2) consists of ciphertext (encryption of m_1), plaintext m_2, and returns an encryption of $(m_1 - m_2)$ (component-wise subtraction).
– **Mult**(ct_1,ct_2) consists of two ciphertexts that contain the encryption of plaintext m_1, m_2, and returns an encryption of $(m_1 \times m_2)$ (component-wise multiplication).
– **MultPlain**(ct_1,m_2) consists of ciphertext (encryption of m_1), plaintext m_2, and returns an encryption of $(m_1 \times m_2)$ (component-wise multiplication).

2.2 Bin Packing Algorithm

Bin packing algorithm involves packing a collection of items into a minimum number of bins, with each bin having a fixed capacity. The goal is to minimize the total number of bins used having restriction that the sum of the weights of the items in each bin should not exceed the bin's capacity (we refer the reader to [36] for more details about bin packing algorithm). In this work, we apply the same algorithm but with additional constraints, which we have discussed in Algorithm 3.

3 Proposed Methodology

The web link(s) retrieval process consists of three servers. With the help of first server, clients performs homomorphic computation over encrypted query, encrypts the location and sends the result to second server. To retrieve the location of web link(s), second server homomorphically decrypts the query vector and sends the encrypted location (if the query decryption result is zero) to the client. Later, with the help of third server, the related web link(s) is retrieved. The steps are discussed as follows.

3.1 Key Sharing Mechanism

Client first shares a secret key (say K') with first server over a secure channel that is used to encrypt the locations (where location can be encrypted using any secure encryption algorithm). Thereafter, client shares the secret key (say K) with second server over a secure channel (where key K is used for homomorphic encryption and decryption).

3.2 Server 1: Existence of entered query and location retrieval

Server 1 is helpful for checking whether the client query q is present in the server or not (as shown in Algorithm 1). First, to perform homomorphic operations, client expands its query vector to include all categories (in our case, a total of five). Later, client encrypts its query vector using BFV encryption (as it offers lower communication complexity with a large dataset) and sends it to the server. Server homomorphically checks the encrypted query from each web link (where the client encrypted vector is subtracted from each link). After performing homomorphic subtraction, the server homomorphically multiplies the resulting vectors and returns the final result along with the corresponding encrypted location to the second server (as shown in Fig. 2). The following toy example illustrates the above observation.

Algorithm 1 Existence of Entered Query with Encrypted Location

Input: Query q, secret keys K and K', Server 1 with stored web links locations $Loc[i, j]$
Output: Homomorphic computed results r_i and encrypted location $\text{Enc}_{K'}(Loc[i, j])$ if q is present
Procedure:
Client Side:
Expand q to form the query vector q' covering all categories
Encrypt the query vector: $\text{Enc}_K(q')$
Send $\text{Enc}_K(q')$ to Server 1
Server 1 Side:
for each web link w_i and corresponding location $Loc[i, j]$ stored on Server 1 **do**
    Compute $d_i = \text{Enc}_K(q') - w_i$ {Homomorphic subtraction}
    Compute $r_i = d_i \cdot d_i$ {Homomorphic multiplication}
    Encrypt the location: $\text{Enc}_{K'}(Loc[i, j])$
end for
Send $\{r_i\}$ and $\{\text{Enc}_{K'}(Loc[i, j])\}$ to Server 2

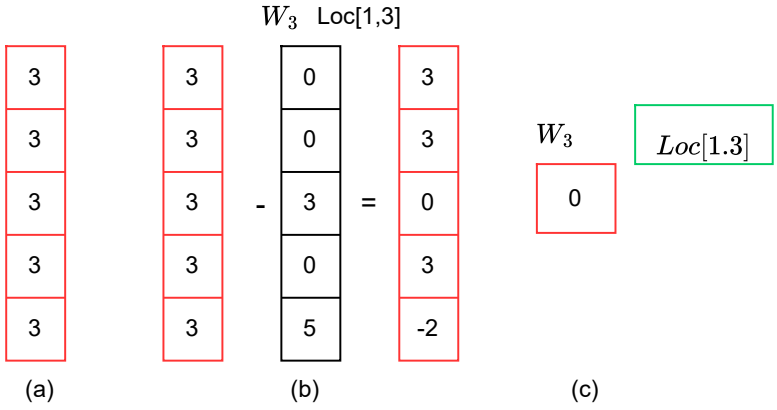

Fig. 2. "Red" color represents encrypted vector using homomorphic encryption, 'Green' color represents encrypted using secure encryption mechanism, and "Black" color represents plaintext vector. (a) indicates an expansion of encrypted query. (b) indicates homomorphic subtraction. (c) indicates homomorphic multiplication of (b)'s results.

Example 1

1. **Client Side:**

(a) The client first expands the query vector to match the total number of categories (in this case, five):

$$q' = [3, 3, 3, 3, 3]$$

(b) The client encrypts the query vector:

$$Enc_K(q') = [Enc_K(3), Enc_K(3), Enc_K(3), Enc_K(3), Enc_K(3)]$$

(c) The encrypted query vector is sent to Server 1.

2. **Server Side:**

(a) Server 1 stores links as vectors. For instance, consider the link W_x:

$$W_x = [0, 0, 3, 0, 5]$$

(b) The server homomorphically subtracts the encrypted query vector from W_x:

$$Enc_K(q' - W_x) = [Enc_K(3), Enc_K(3), Enc_K(0), Enc_K(3), Enc_K(-2)]$$

(c) The server homomorphically multiplies the resulting encrypted vector:

$$Enc_K\left(3 \times 3 \times 0 \times 3 \times (-2)\right) = Enc_K(0)$$

(d) The encrypted result is sent to the server 2 with encrypted location (say $Enc_{K'}(Loc[i, j])$).

3.3 Server 2: Retrieval of relevant web links locations

After receiving the encrypted results and encrypted locations, Server 2 homomorphically decrypts the results (as shown in Algorithm 2). If decryption results is zero then Server 2 sends the encrypted location to the client. The following toy example illustrates the above observations.

1. **Server Side:**

(a) The client decrypts the result:

$$Dec_K(Enc_K(0)) = 0$$

(b) Since the decryption result is 0, the client confirms that the query is present in the link W_x, and sends the encrypted location to the client $(Enc_{K'}(Loc([i, j]))$.

2. **Client Side:**

(a) Client decrypts the encrypted location as follows.

$$Dec_{K'}(Enc_{K'}(Loc[i, j])) = Loc[i, j]$$

Algorithm 2 Server 2: Decryption and Location Retrieval

Input: Homomorphic results $\{r_i\}$, decryption key K, encrypted locations $\{\text{Enc}_{K'}(Loc[i,j])\}$
Output: Decrypted location(s) ($Loc[i,j]$) if query q is present
Server 2 Side:
Decrypt the results: $\{\text{Dec}_K(r_i)\}$
for each decrypted result $\text{Dec}_K(r_i)$ **do**
  if $\text{Dec}_K(r_i) = 0$ **then**
    Output: Query q is present; send corresponding encrypted location $\text{Enc}_{K'}(Loc[i,j])$ to the client
  end if
  if No $\text{Dec}_K(r_i) = 0$ found **then**
    Output: `abort` {Query not present; restart with new query}
  end if
end for
Client Side:
Decrypt the locations: $\{\text{Dec}(\text{Enc}_{K'}(Loc[i,j]))\}$, that is, $Loc[i,j]$

3.4 Server 3: Relevant links retrieval

This section helps retrieve the relevant web links. First, note that all links are stored on Server 3. As links can be of any size. Thus, to reduce the wastage of memory (as random way of storage may lead to usage of more memory), we are proposing a novel restricted bin packing algorithm (RBA) (as shown in Algorithm 3). The main motivation of RBA is to divide each bin into an equal number of slots that is useful for performing homomorphic operations (as performed in Section 3.1). Client first encrypts the query vector, that is, encryption of 0's except at desired location (where the web link is present), that is, encryption of 1, and sends it to the server. Server homomorphically multiplies the query vector with the bin, and then homomorphic addition is performed. The obtained result is again sent back to the client. Client decrypts it and achieves the desired link(s) (as shown in Algorithm 4). The above observation is demonstrated by the following toy example. Fig. 3 represents the pictorial overview of above discussion.

1. **Client Side:**
   (a) The client encrypts the query vector (as the desired link is present at third slot of first bin) :
   $$q' = [0, 0, 1, 0]$$
   where the total number of slots in each bin is four.
   (b) The client encrypts the query vector:
   $$Enc_K(q') = [Enc_K(0), Enc_K(0), Enc_K(1), Enc_K(0)]$$
   (c) The encrypted query vector is sent to the server.
2. **Server Side:**

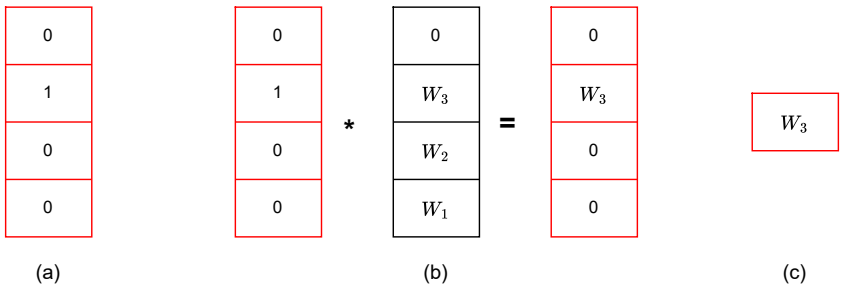

Fig. 3. "Red" color represents encrypted vector, and "Black" color represents plaintext vector. (a) indicates an expansion of encrypted query (where the desired location contains 1 and other contains 0's). (b) indicates homomorphic multiplication. (c) indicates homomorphic addition of (b)'s results.

(a) The first bin contains the following items:

$$\text{Bin 1: } W_1, W_2, W_3, 0$$

(b) The server performs homomorphic multiplication of the query vector with each item in the bin:

$$Enc_K(0 \times W_1), Enc(0 \times W_2), Enc(1 \times W_3), Enc(0 \times 0)$$

which results in:

$$[Enc_K(0), Enc_K(W_0), Enc_K(W_3), Enc_K(0)]$$

(c) The server then performs homomorphic addition of the results:

$$Enc_K(0 + 0 + W_3 + 0) = Enc_K(W_3)$$

(d) The encrypted result $Enc_K(W_3)$ is sent back to the client.

3. **Client Side:**

(a) The client decrypts the received result:

$$Dec(Enc_K(W_3)) = W_3$$

(b) The client retrieves W_3.

4 Performance Evaluation

In this section, we evaluate the performance of our protocol and compare it with the state-of-the-art scheme in [26]. The protocol proposed in [26] used 65536 keywords for document retrieval, which shows that for running a single query, a client needs to spend 1.64 dollars. Our implementation setup and results show that a client can retrieve the desired web link(s) without spending any amount in dollars. In the next section, we discuss the implementation setup and its results.

Algorithm 3 Restricted Bin Packing Algorithm (RBA)

Input: Set of n links with their weights $\gamma = \{\gamma_1, \gamma_2, \ldots, \gamma_n\}$.
Output: Minimum number of bins required.
Step 1: Initialize
$Max\_link\_size \leftarrow \max(\gamma)$
$Min\_link\_size \leftarrow \min(\gamma)$
Step 2: Compute bin capacity
$Bin\_capacity \leftarrow Max\_link\_size + \lceil \frac{Min\_link\_size}{2} \rceil$
Step 3: Compute possible number of slots in each bin
$Slots\_per\_bin \leftarrow \left\lceil \frac{Max\_link\_size}{Min\_link\_size} \right\rceil$
Step 4: Initialize empty bins and current bin capacity
$bins \leftarrow \emptyset$ {List of bins}
$current\_bin\_capacity \leftarrow Bin\_capacity$
Step 5: For each item $\gamma_i \in \gamma$:
for each item $\gamma_i \in \gamma$ **do**
  if $\gamma_i < current\_bin\_capacity$ **then**
    Place γ_i in the current bin
    Update $current\_bin\_capacity \leftarrow current\_bin\_capacity - \gamma_i$
  else if $\gamma_i = current\_bin\_capacity$ **then**
    Start a new bin and place γ_i in the new bin
    Reset $current\_bin\_capacity \leftarrow Bin\_capacity - \gamma_i$
  else
    Start a new bin
    Place γ_i in the new bin
    Reset $current\_bin\_capacity \leftarrow Bin\_capacity - \gamma_i$
  end if
end for
Step 6: Handle incomplete bins
if $current\_bin\_capacity > 0$ **then**
  Divide the remaining capacity into R_s random slots
  $Total\_slots\_in\_bin \leftarrow R_s +$ Number of occupied slots
  Fill the empty slots with the number 0
end if
Step 7: Output the number of bins used
Output $\#bins$

4.1 Implementation Setup

The proof-of-concept code for our observation is written in Jupyter notebook (python version 3.9.10) using TenSeal library. Jupyter notebook is installed over Intel(R) Core(TM) i5-10500 CPU @ 3.10GHz 3.10 GHz. The installed memory is 16GB.

4.2 Implementation Results

Data collection: We have collected the web links from Wikipedia. The total number of web links are 34930.[4] The distribution of links are as follows: 16365

[4] A single link can discuss about multiple categories.

Algorithm 4 Web Link Retrieval Algorithm

Input: Location of web link $Loc[i, j]$.
Output: Web link corresponding to location $Loc[i, j]$.
Procedure:
Client Side:
Create a query vector q' with 0's except at the desired location $Loc[i, j]$, where it is set to 1.
Encrypt the query vector $\text{Enc}_K(q')$.
Send $\text{Enc}_K(q')$ to the server.
Server Side:
Divide the bins as per the Restricted Bin Packing Algorithm (RBA) to store web links.
for each bin b_i that stores web links **do**
    Compute $d_i = \text{Enc}_K(q') \cdot b_i$. {Homomorphic multiplication of the query vector with the bin.}
end for
Perform homomorphic addition on the results: $r = \sum d_i$. {Homomorphic addition of the multiplied results.}
Send the final result $\text{Enc}_K(r)$ back to the client.
Client Side:
Decrypt $\text{Enc}_K(r)$ to obtain the web link.
Output: Retrieve the desired web link.

links discuss about category L; 26111 links discuss about category G; 10898 links discuss about category B; 11781 links discuss about category T; and 10610 links discuss about the category Q.[5]

Data preprocessing over Server 1, Server 2, and Server 3: This stage discuss about the implementation of Server 1, Server 2, and Server 3. Server 3 stores the web link(s) in accordance to RBA algorithm whereas Server 1 arranges the category of web link(s), and Server 2 stores the computed results from Server 1. We execute our program once and the execution time for the development of Server 1, Server 2, and Server 3 are 29.13 seconds, 1631.65 and 47.29 seconds respectively. Total number of bins are 18854 whereas each bin contain 9 slots.

Implementation of protocol: We discuss the Server 2 and Server 3 execution time for retrieving the desired web link(s) corresponding to client's query. We execute our program over several iterations. Table [1, 2] indicates the total execution time for retrieving location of desired web link(s) from Server 2, and desired web link(s) from Server 3. Implementation results show that execution time is high whenever a client needs to retrieve more number of web link(s).[6] Moreover,

[5] we keep our dataset publicly available at https://github.com/akshit-aggarwal/Empowering-Privacy-A-Zero-Cost-Protocol-for-Concealing-LGBTQ-Search-Queries.
[6] Since it utilizes linear search.

implementation shows that a client can retrieve the desired web link(s) with zero cost (specifically in terms of dollars).

Iteration(s)	Server 2 execution time (in sec(s))				
	L	G	B	T	Q
1	14.9300	0.0200	0.0200	17.1600	15.4300
10	16.8500	0.2500	0.2600	17.2500	16.8600
100	18.4500	2.3400	1.5000	18.5500	19.5800
1000	33.1100	16.1400	16.8600	33.8000	34.4600
10000	350.2000	163.1400	533.3700	483.8500	571.2000

Table 1. Server 2 execution time for location retrieval with different variation of links. Here Iteration(s) shows total number of link(s) retrieval.

Iteration(s)	Server 3 execution time (in sec(s))				
	L	G	B	T	Q
1	0.1241	0.1216	0.1321	0.1304	0.1190
10	0.2198	0.1711	0.1463	0.1749	0.1746
100	0.4067	0.3695	0.4351	0.3742	0.4284
1000	3.0884	2.6792	2.6519	4.0001	3.7355
10000	29.6988	28.6350	28.4684	28.6017	28.5477

Table 2. Server 3 execution time for link retrieval related to category (L/G/B/T/Q). Here Iteration(s) shows total number of link(s) retrieval.

5 Security and Correctness

This section discusses the security and correctness of our work. First, we discuss the security of our work with respect to encryption scheme and web link(s) retrieval process. Later, we check the correctness of our work by comparing it with some other existing literature.

5.1 Security:

Our designed protocol guarantee query privacy for its clients. Our work uses the BFV scheme to encrypt the client's query. Thus, the construction of BFV is based on ring learning with error (that is hard for any adversary to break). Moreover, BFV also provides semantic security. The web link(s) retrieval process is all anonymous. Thus, server will never get information about the query, retrieved web link(s), or its location.

5.2 Correctness:

The correctness of our work is computed in various steps. In first step, we discuss the results retrieved from Server 1. Later, we compare our work with some other existing works [25,26,29,32] and show the optimisation of our protocol. Our work shows that if the entered query by client is present on the server, then decrypted result by client is zero; otherwise, it will return a non-zero value. Thus, it indicates that our proposed work correctly checks whether the entered query is present on the server or not.
Moreover, we also verified our work with some other literature that discusses the optimisation of problems while designing any PIR-based protocol (as discussed in Table 3).
Thus, Table 3 shows the correctness of our work in terms of optimising the mentioned problems while designing any PIR-based protocol.

Literature	Problem(s)	Our work
[25]	Query leakage due to ASPE scheme.	No leakage of query.
[26]	6 interaction rounds between client and servers, and Oblivious transfer in round 3.	5 interaction rounds , and fully PIR based scheme in each step.
[29]	High number of additional HE operations due to Fermat's little theorem, One's complement.	No need of additional operations except of HE multiplication, subtraction, and additions.
[32]	Need high client storage.	No need of client storage except client's query.

Table 3. Here, the problems of existing works are discussed and compared with our work.

6 Limitations

The proposed work only discusses the security of FHE (that is, BFV encryption), but it does not verify the designed protocol concerning side-channel attacks (or any cryptographic attacks). The proposed work cannot retrieve the results for more than one query simultaneously.

7 Conclusion

In this work, we design a novel protocol that maintains the privacy of LGBTQ groups. Our designed protocol uses the less number of homomorphic operations compared to other existing schemes. Moreover, our work also guarantees the correct query result without any query leakage. Thus, from our proposed work, the category of LGBTQ can easily retrieve the relevant web link(s) without spending any amount in dollars.

References

1. J. Allen, "Recent cyber attacks & data breaches in 2021," https://purplesec.us/recent-cyber-security-attacks/, 2021, accessed: 2022-12-14.
2. I. Vojinovic, "Data breach statistics that will make you think twice before filling out an online form," https://dataprot.net/statistics/data-breach-statistics/, 2022, accessed: 2023-07-23.
3. P. Grubbs, M.-S. Lacharité, B. Minaud, and K. G. Paterson, "Learning to Reconstruct: Statistical Learning Theory and Encrypted Database Attacks," in *Proceedings of the IEEE Symposium on Security and Privacy (SP)*, 2019, pp. 1067–1083.
4. W. K. Wong, D. W.-l. Cheung, B. Kao, and N. Mamoulis, "Secure kNN Computation on Encrypted Databases," in *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*, 2009, pp. 139–152.
5. H.-J. Kim, H. Lee, Y.-K. Kim, and J.-W. Chang, "Privacy-Preserving kNN Query Processing Algorithms via Secure Two-Party Computation over Encrypted Database in Cloud Computing," *The Journal of Supercomputing*, vol. 78, no. 7, pp. 9245–9284, 2022.
6. Y. Zheng, R. Lu, S. Zhang, J. Shao, and H. Zhu, "Achieving Practical and Privacy-Preserving kNN Query over Encrypted Data," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–13, 2024.
7. W. Wu, J. Liu, H. Rong, H. Wang, and M. Xian, "Efficient k-Nearest Neighbor Classification over Semantically Secure Hybrid Encrypted Cloud Database," *IEEE Access*, vol. 6, pp. 41 771–41 784, 2018.
8. S. Vivek, S. Murthy, and D. Kumaraswamy, "Integer Polynomial Recovery from Outputs and its Application to Cryptanalysis of a Protocol for Secure Sorting," *Journal of Mathematical Cryptology*, vol. 16, no. 1, pp. 251–277, 2022.
9. D. Demmler, P. Rindal, M. Rosulek, and N. Trieu, "PIR-PSI: Scaling Private Contact Discovery," *Proceedings on Privacy Enhancing Technologies*, p. 159–178, 2018.
10. E. Fung, G. Kellaris, and D. Papadias, "Combining Differential Privacy and PIR for Efficient Strong Location Privacy," in *International Symposium on Spatial and Temporal Databases*, 2015, pp. 295–312.
11. S. Angel and S. Setty, "Unobservable Communication over Fully Untrusted Infrastructure," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 551–569.
12. P. Mittal, F. Olumofin, C. Troncoso, N. Borisov, and I. Goldberg, "PIR-Tor: Scalable Anonymous Communication using Private Information Retrieval," in *20th USENIX security symposium (USENIX security 11)*, 2011, pp. 1–31.
13. A. Singh and K. Chatterjee, "SecEVS: Secure Electronic Voting System using Blockchain Technology," in *Proceedings of the International Conference on Computing, Power and Communication Technologies (GUCON)*, 2018, pp. 863–867.
14. V. Agate, A. De Paola, P. Ferraro, G. L. Re, and M. Morana, "SecureBallot: A Secure Open Source E-Voting System," *Journal of Network and Computer Applications*, vol. 191, pp. 103–165, 2021.
15. A. Vadapalli, F. Bayatbabolghani, and R. Henry, "You May Also Like... Privacy: Recommendation Systems Meet PIR," *Proceedings on Privacy Enhancing Technologies*, pp. 30–53, 2021.
16. A. K. Lenstra and M. S. Manasse, "Factoring with two large primes," in *Advances in Cryptology—EUROCRYPT'90: Workshop on the Theory and Application of Cryptographic Techniques*, 1991, pp. 72–82.

17. A. M. Odlyzko, "Discrete logarithms in finite fields and their cryptographic significance," in *Workshop on the Theory and Application of of Cryptographic Techniques*, 1984, pp. 224–314.
18. R. Ostrovsky and W. E. Skeith III, "A Survey of Single-Database Private Information Retrieval: Techniques and Applications," in *International Workshop on Public Key Cryptography*, 2007, pp. 393–411.
19. B. Chor and N. Gilboa, "Computationally Private Information Retrieval," in *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, 1997, pp. 304–313.
20. E. Kushilevitz and R. Ostrovsky, "Replication is not needed: Single Database, Computationally-Private Information Retrieval," in *Proceedings 38th annual symposium on foundations of computer science*, 1997, pp. 364–373.
21. I. Goldberg, "Improving the Robustness of Private Information Retrieval," in *IEEE Symposium on Security and Privacy (SP'07)*, 2007, pp. 131–148.
22. X. Yi, M. G. Kaosar, R. Paulet, and E. Bertino, "Single-Database Private Information Retrieval From Fully Homomorphic Encryption," *IEEE Transactions on Knowledge and Data Engineering*, vol. 25, no. 5, pp. 1125–1134, 2012.
23. R. Henry, "Polynomial Batch Codes for Efficient IT-PIR," *Cryptology ePrint Archive*, 2016. [Online]. Available: https://eprint.iacr.org/2016/598
24. S. Angel, H. Chen, K. Laine, and S. Setty, "PIR with Compressed Queries and Amortized Query Processing," in *IEEE Symposium on Security and Privacy (SP)*, 2018, pp. 962–979.
25. J. Fu, N. Wang, B. Cui, and B. K. Bhargava, "A Practical Framework for Secure Document Retrieval in Encrypted Cloud File Systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 5, pp. 1246–1261, 2021.
26. I. Ahmad, L. Sarker, D. Agrawal, A. El Abbadi, and T. Gupta, "Coeus: A System for Oblivious Document Ranking and Retrieval," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021, pp. 672–690.
27. D. Kogan and H. Corrigan-Gibbs, "Private Blocklist Lookups with Checklist," in *30th USENIX security symposium (USENIX Security 21)*, 2021, pp. 875–892.
28. H. Corrigan-Gibbs, A. Henzinger, and D. Kogan, "Single-Server Private Information Retrieval with Sublinear Amortized Time," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2022, pp. 3–33.
29. I. Ahmad, D. Agrawal, A. E. Abbadi, and T. Gupta, "Pantheon: Private Retrieval from Public Key-Value Store," *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 643–656, 2022.
30. S. Colombo, K. Nikitin, H. Corrigan-Gibbs, D. J. Wu, and B. Ford, "Authenticated Private Information Retrieval," in *32nd USENIX security symposium (USENIX Security 23)*, 2023, pp. 3835–3851.
31. A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan, "One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 3889–3905.
32. A. Davidson, G. Pestana, and S. Celi, "FrodoPIR: Simple, Scalable, Single-Server Private Information Retrieval," in *Proceedings on Privacy Enhancing Technologies*, 2023, pp. 365–383.
33. M. Zhou, A. Park, E. Shi, and W. Zheng, "Piano: Extremely Simple, Single-Server PIR with Sublinear Server Computation," *Cryptology ePrint Archive*, 2023. [Online]. Available: https://eprint.iacr.org/2023/452
34. O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography," *Journal of the ACM (JACM)*, vol. 56, no. 6, pp. 1–40, 2009.

35. J. Fan and F. Vercauteren, "Somewhat Practical Fully Homomorphic Encryption," *Cryptology ePrint Archive*, 2012. [Online]. Available: https://ia.cr/2012/144
36. E. G. Coffman, J. Csirik, G. Galambos, S. Martello, D. Vigo *et al.*, "Bin Packing Approximation Algorithms: Survey and Classification." in *Handbook of combinatorial optimization*, 2013, pp. 455–531.